Building Diabetes Prediction Models

Model 1 Preparation

```
df_multilog <- diabetes |>
  select(diabetes_012, high_bp, high_chol, chol_check, bmi, smoker, stroke,
         heart_diseaseor_attack, phys_activity, fruits, veggies, hvy_alcohol_consump,
         any_healthcare, no_docbc_cost, gen_hlth, ment_hlth, phys_hlth, diff_walk,
         sex, age, education, income)
df_multilog
```

```
## # A tibble: 253,680 x 22
##    diabetes_012 high_bp high_chol chol_check   bmi smoker stroke
##    <fct>        <fct>   <fct>     <fct>      <dbl> <fct>  <fct>
##  1 No Diabetes  Yes     Yes       Yes           40 Yes    No
##  2 No Diabetes  No      No        No            25 Yes    No
##  3 No Diabetes  Yes     Yes       Yes           28 No     No
##  4 No Diabetes  Yes     No        Yes           27 No     No
##  5 No Diabetes  Yes     Yes       Yes           24 No     No
##  6 No Diabetes  Yes     Yes       Yes           25 Yes    No
##  7 No Diabetes  Yes     No        Yes           30 Yes    No
##  8 No Diabetes  Yes     Yes       Yes           25 Yes    No
##  9 Diabetes     Yes     Yes       Yes           30 Yes    No
## 10 No Diabetes  No      No        Yes           24 No     No
## # i 253,670 more rows
## # i 15 more variables: heart_diseaseor_attack <fct>, phys_activity <fct>,
## #   fruits <fct>, veggies <fct>, hvy_alcohol_consump <fct>,
## #   any_healthcare <fct>, no_docbc_cost <fct>, gen_hlth <ord>, ment_hlth <dbl>,
## #   phys_hlth <dbl>, diff_walk <fct>, sex <fct>, age <ord>, education <ord>,
## #   income <ord>
```

train/test split

```
# using createDataPartition because diabetes_012 is unbalanced based on EDA.
train_test_split <- createDataPartition(df_multilog$diabetes_012, p = 0.8, list = FALSE)
train <- df_multilog[train_test_split, ]
test  <- df_multilog[-train_test_split, ]

y_test <- test$diabetes_012
```

F1 function for multiclass

```r
multi_f1 <- function(true, pred) {
  # Convert to factors
  true <- as.factor(true)
  pred <- as.factor(pred)

  # make sure all classes are taken into consideration (bc diabetes=1 is rare)
  class <- union(levels(true), levels(pred))
  true <- factor(true, levels = class)
  pred <- factor(pred, levels = class)

  # A vector to store F1 for each class
  multi_f1_vec <- numeric(length(class))

  # Loop over each class
  for (j in seq_along(class)) {
    i <- class[j]
    tp <- sum(true == i & pred == i)
    fn <- sum(true == i & pred != i)
    fp <- sum(true != i & pred == i)
    rec  <- if((tp + fn) == 0) 0 else tp / (tp + fn)
    prec <- if((tp + fp) == 0) 0 else tp / (tp + fp)

    if(prec + rec == 0) {
      multi_f1_vec[j] <- 0
    } else {
      multi_f1_vec[j] <- 2 * prec * rec / (prec + rec)
    }
  }
    mean(multi_f1_vec)
}
```

Model 1: Multinomial Logistic Regression

```r
model1 <- multinom(diabetes_012 ~ ., data = train, trace = FALSE)
```

```r
model1_pred <- predict(model1, newdata = test, type = "class")
model1_prob <- predict(model1, newdata = test, type = "prob")

model1_accuracy <- mean(model1_pred == y_test)
cat("Accuracy is:", model1_accuracy, "\n")
```

```
## Accuracy is: 0.8486843
```

```r
model1_macro_f1  <- multi_f1(y_test, model1_pred)
cat("Macro F1 score is:", model1_macro_f1, "\n")
```

```
## Macro F1 score is: 0.3948124
```

```r
model1__macro_auc <- as.numeric(multiclass.roc(response = y_test, predictor = model1_prob)$auc)
cat("Macro AUC is:", model1__macro_auc, "\n")
```

```
## Macro AUC is: 0.7007838
```

ROC

```r
cols <- c("blue", "green", "pink")

# Compute one-vs-rest ROC and AUC for each class
model1_roc1 <- roc(as.numeric(y_test == levels(y_test)[1]), model1_prob[, levels(y_test)[1]])
```

```
## Setting levels: control = 0, case = 1

## Setting direction: controls < cases
```

```r
model1_roc2 <- roc(as.numeric(y_test == levels(y_test)[2]), model1_prob[, levels(y_test)[2]])
```

```
## Setting levels: control = 0, case = 1
## Setting direction: controls < cases
```

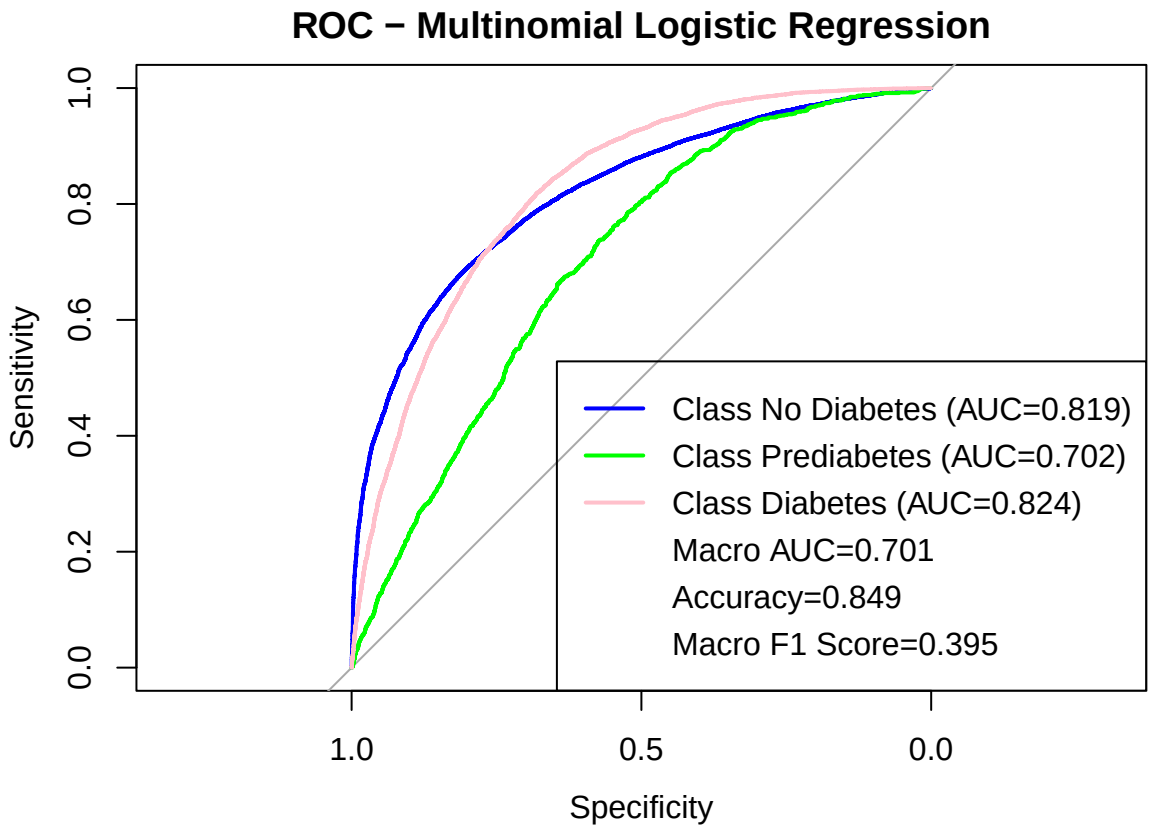
```r
model1_roc3 <- roc(as.numeric(y_test == levels(y_test)[3]), model1_prob[, levels(y_test)[3]])
```

```
## Setting levels: control = 0, case = 1
## Setting direction: controls < cases
```

```r
# Plot ROC
plot(model1_roc1, col = cols[1], main = "ROC - Multinomial Logistic Regression")
lines(model1_roc2, col = cols[2])
lines(model1_roc3, col = cols[3])

# Compute class AUCs
model1_class_aucs <- sapply(levels(y_test), function(cl) round(auc(roc(as.numeric(y_test == cl), model1
```

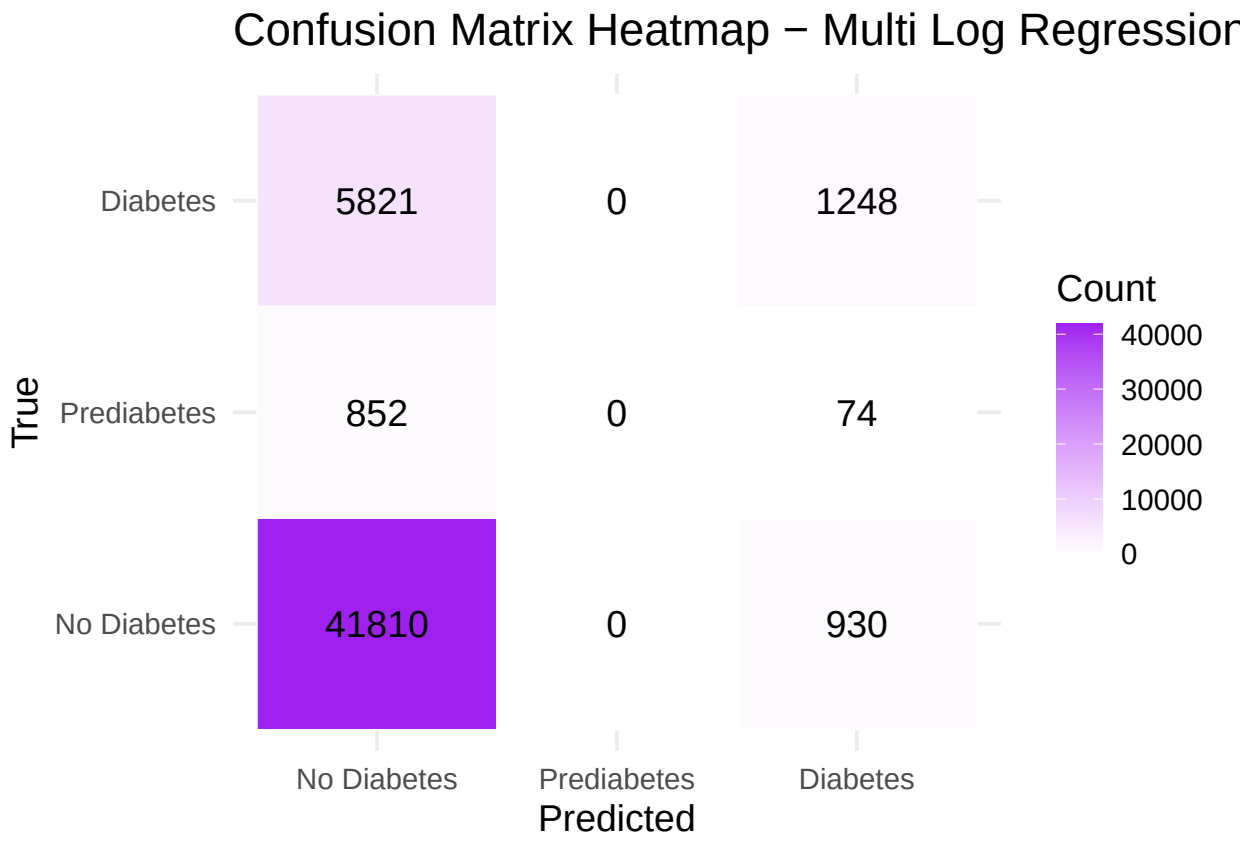
```
## Setting levels: control = 0, case = 1
## Setting direction: controls < cases

## Setting levels: control = 0, case = 1

## Setting direction: controls < cases

## Setting levels: control = 0, case = 1

## Setting direction: controls < cases
```

```
# Add legend with class AUCs + macro AUC
legend("bottomright",
       legend = c(paste0("Class ", levels(y_test), " (AUC=", model1_class_aucs, ")"),
                  paste0("Macro AUC=", round(model1__macro_auc, 3)),
                  paste0("Accuracy=", round(model1_accuracy, 3)),
                  paste0("Macro F1 Score=", round(model1_macro_f1, 3))),
       col = c(cols, rep(NA, 3)),
       lwd = c(rep(2, 3), rep(NA, 3)))
```



Confusion matrix heatmap

```
model1_cm <- table(True = y_test, Pred = model1_pred)
model1_cm_df <- as.data.frame(model1_cm) |>
  rename(True = True, Predicted = Pred, Count = Freq)

ggplot(model1_cm_df, aes(x = Predicted, y = True, fill = Count)) +
  geom_tile(color = "white") +
  geom_text(aes(label = Count), color = "black", size = 5) +
  scale_fill_gradient(low = "white", high = "purple") +
  labs(title = "Confusion Matrix Heatmap - Multi Log Regression", x = "Predicted", y = "True") +
  theme_minimal(base_size = 14)
```



Model 2 Preparation

```
df_ranger <- diabetes |>
  select(diabetes_2, high_bp, high_chol, chol_check, bmi, smoker, stroke,
         heart_diseaseor_attack, phys_activity, fruits, veggies, hvy_alcohol_consump,
         any_healthcare, no_docbc_cost, gen_hlth, ment_hlth, phys_hlth, diff_walk,
         sex, age, education, income)

df_ranger$diabetes_2 <- factor(df_ranger$diabetes_2,
                               levels = c(0, 1),
                               labels = c("No_Diabetes", "Diabetes"))
df_ranger
```

```
## # A tibble: 253,680 x 22
##    diabetes_2  high_bp high_chol chol_check   bmi smoker stroke
##    <fct>       <fct>   <fct>     <fct>      <dbl> <fct>  <fct>
##  1 No_Diabetes Yes     Yes       Yes           40 Yes    No
##  2 No_Diabetes No      No        No            25 Yes    No
##  3 No_Diabetes Yes     Yes       Yes           28 No     No
##  4 No_Diabetes Yes     No        Yes           27 No     No
##  5 No_Diabetes Yes     Yes       Yes           24 No     No
##  6 No_Diabetes Yes     Yes       Yes           25 Yes    No
##  7 No_Diabetes Yes     No        Yes           30 Yes    No
```

```
##  8 No_Diabetes Yes     Yes       Yes           25 Yes    No
##  9 Diabetes    Yes     Yes       Yes           30 Yes    No
## 10 No_Diabetes No      No        Yes           24 No     No
## # i 253,670 more rows
## # i 15 more variables: heart_diseaseor_attack <fct>, phys_activity <fct>,
## #   fruits <fct>, veggies <fct>, hvy_alcohol_consump <fct>,
## #   any_healthcare <fct>, no_docbc_cost <fct>, gen_hlth <ord>, ment_hlth <dbl>,
## #   phys_hlth <dbl>, diff_walk <fct>, sex <fct>, age <ord>, education <ord>,
## #   income <ord>
```

Model 2: Binary Ranger/RandomForest Model

Data splitting

```
set.seed(1)

index <- createDataPartition(df_ranger$diabetes_2, p = 0.8, list = FALSE)
train_data <- df_ranger[index, ]
test_data <- df_ranger[-index, ]

y_test <- test_data$diabetes_2
```

Train Binary Random Forest Model (using ranger)

```
rf_model <- ranger(
  dependent.variable.name = "diabetes_2",
  data = train_data,
  num.trees = 500,
  importance = "impurity",
  seed = 1,
  verbose = TRUE,
  probability = TRUE
)
```

```
## Growing trees.. Progress: 45%. Estimated remaining time: 38 seconds.
## Growing trees.. Progress: 88%. Estimated remaining time: 8 seconds.
```

Prediction and Metrics

```
pred <- predict(rf_model, data = test_data, type = "response")

# Probability of Diabetes (positive class)
model_prob <- pred$predictions[, "Diabetes"]

# Class predictions = class with highest probability
model_pred <- colnames(pred$predictions)[ max.col(pred$predictions) ]
model_pred <- factor(model_pred, levels = levels(y_test))

# Confusion matrix
cm <- confusionMatrix(model_pred, y_test)
```

```r
# Accuracy
model_accuracy <- as.numeric(cm$overall['Accuracy'])

# F1 for Diabetes
f1_diabetes <- as.numeric(cm$byClass['F1'])
# F1 for No_Diabetes
cm_rev <- confusionMatrix(model_pred, y_test, positive = "No_Diabetes")
f1_no_diabetes <- as.numeric(cm_rev$byClass['F1'])
# Macro F1
model_macro_f1 <- mean(c(f1_diabetes, f1_no_diabetes))

# AUC for Diabetes
model_roc <- roc(response = y_test, predictor = model_prob)
```

```
## Setting levels: control = No_Diabetes, case = Diabetes
```

```
## Setting direction: controls < cases
```

```r
model_auc <- as.numeric(model_roc$auc)

cat("Accuracy is:     ", round(model_accuracy, 4), "\n")
```

```
## Accuracy is:      0.8523
```

```r
cat("Macro F1 score is:", round(model_macro_f1, 4), "\n")
```

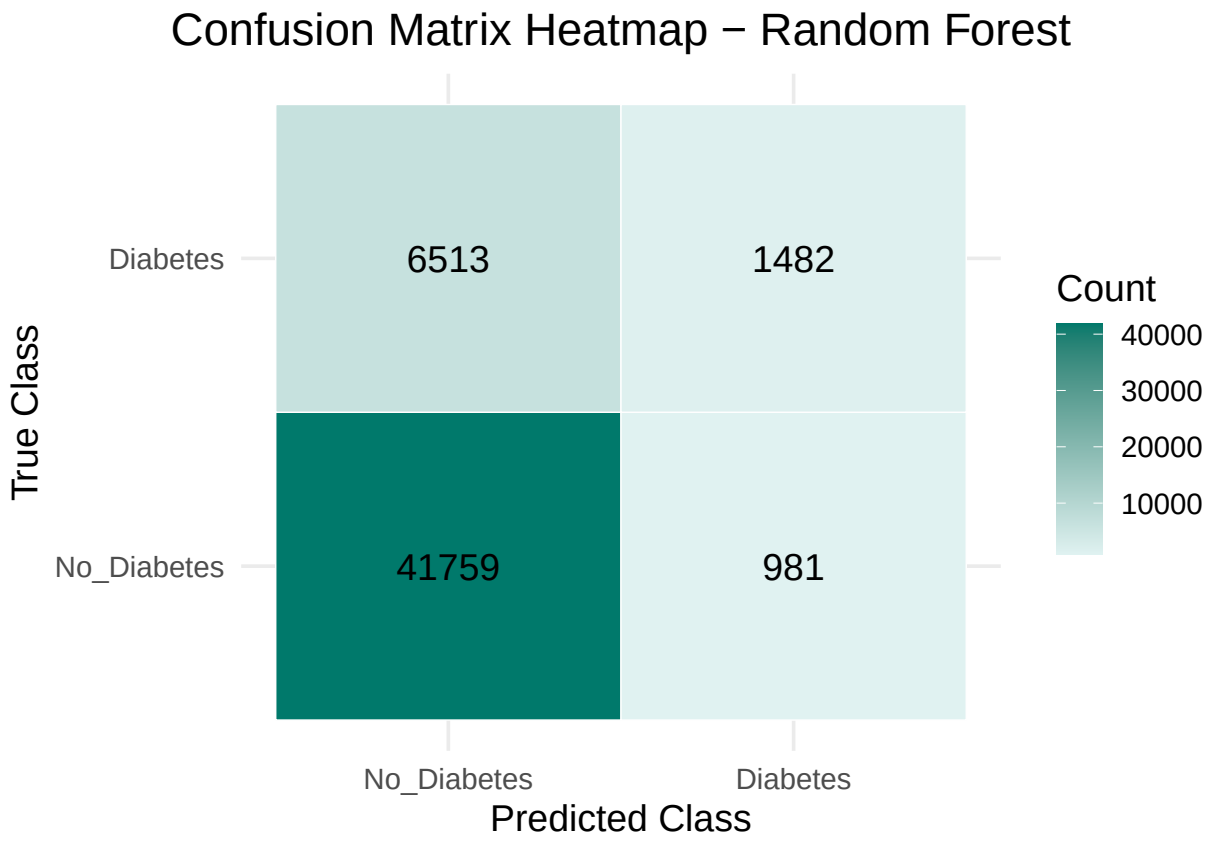
```
## Macro F1 score is: 0.9177
```

```r
cat("ROC AUC (Binary) is:", round(model_auc, 4), "\n")
```

```
## ROC AUC (Binary) is: 0.8195
```

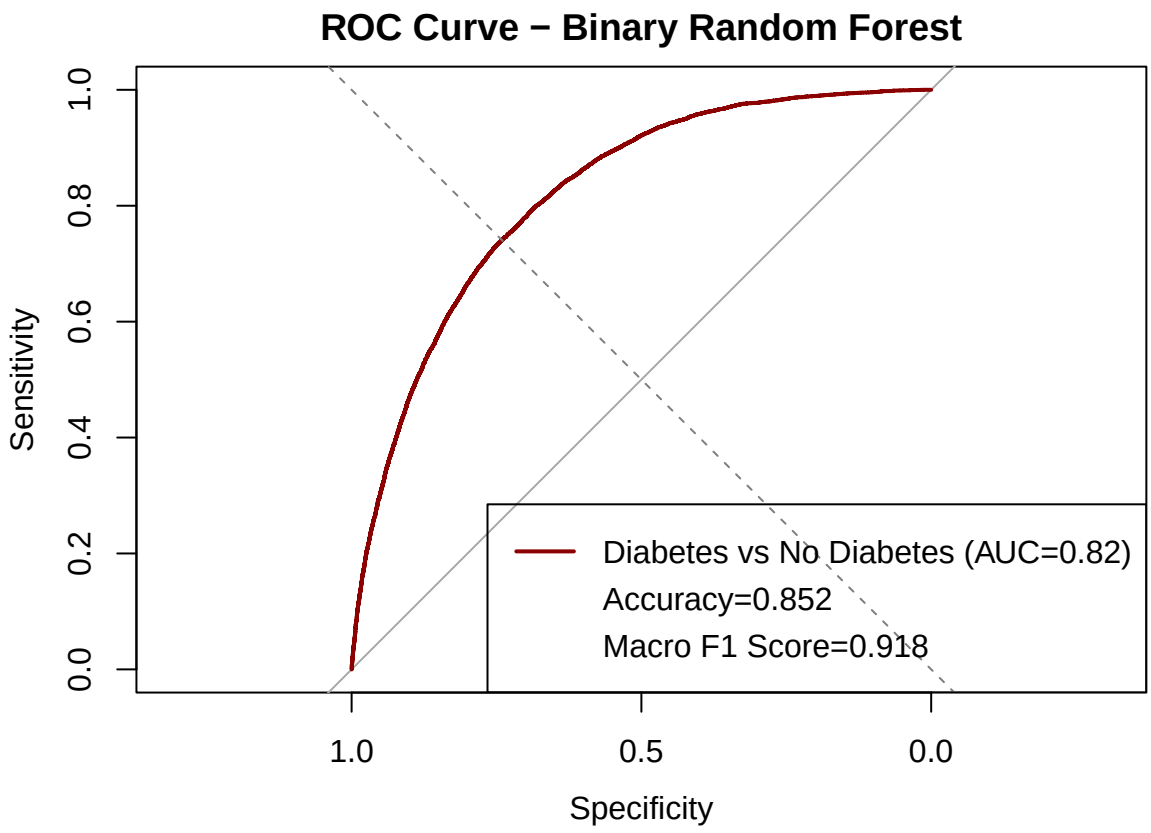
Confusion matrix heatmap

```r
model_cm_table <- cm$table

model_cm_df <- as.data.frame(model_cm_table) |>
  rename(True = Reference, Predicted = Prediction, Count = Freq)

p_heatmap <- ggplot(model_cm_df, aes(x = Predicted, y = True, fill = Count)) +
  geom_tile(color = "white") +
  geom_text(aes(label = Count), color = "black", size = 5) +
  scale_fill_gradient(low = "#e0f2f1", high = "#00796b") +
  labs(
    title = "Confusion Matrix Heatmap - Random Forest",
    x = "Predicted Class",
    y = "True Class"
  ) +
  theme_minimal(base_size = 14) +
  theme(plot.title = element_text(hjust = 0.5))

print(p_heatmap)
```



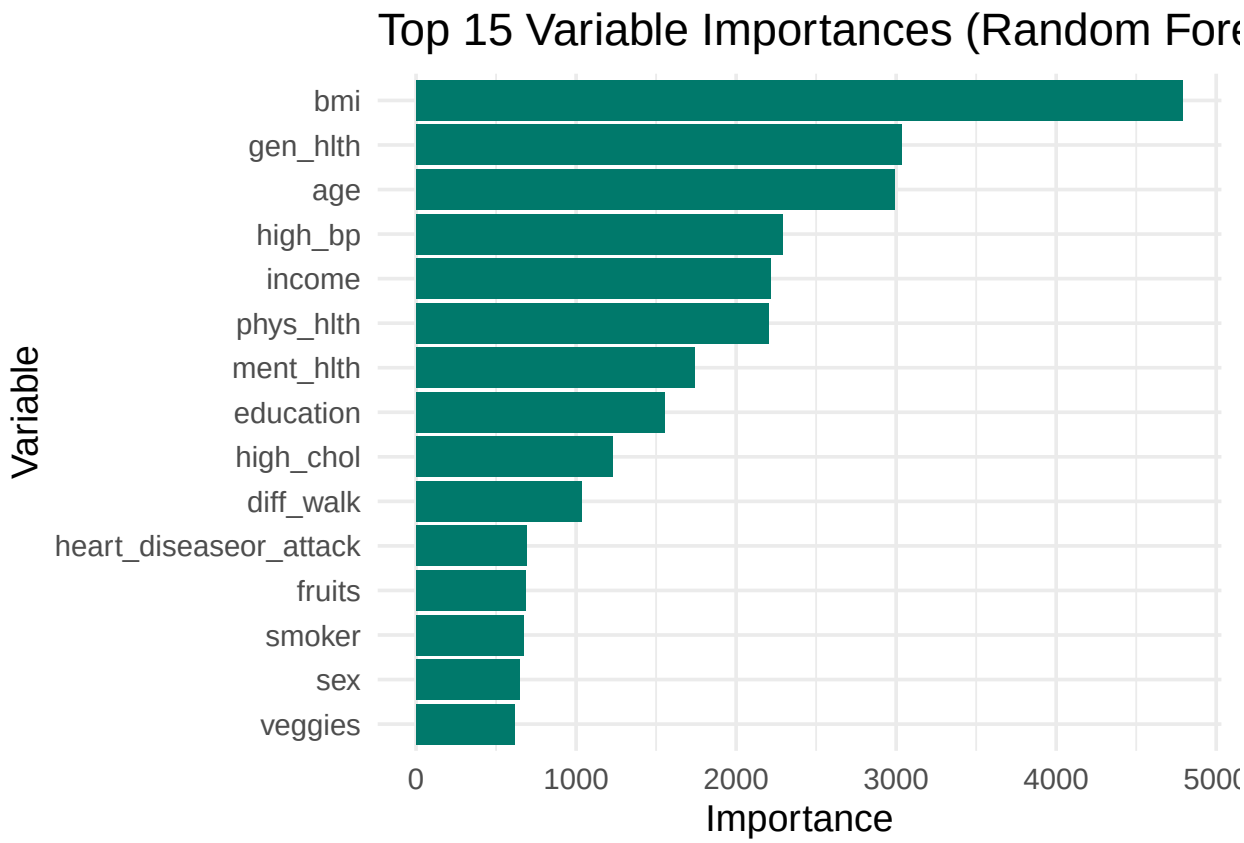
ROC curve

```
plot(model_roc,
     col = "darkred",
     main = "ROC Curve - Binary Random Forest",
     lwd = 2)

abline(a = 0, b = 1, lty = 2, col = "grey50")

legend(
  "bottomright",
  legend = c(
    paste0("Diabetes vs No Diabetes (AUC=", round(model_auc, 3), ")"),
    paste0("Accuracy=", round(model_accuracy, 3)),
    paste0("Macro F1 Score=", round(model_macro_f1, 3))
  ),
  col = c("darkred", rep(NA, 2)),
  lwd = c(2, rep(NA, 2))
)
```



Variable importance

```
importance_df <- data.frame(
  Variable = names(rf_model$variable.importance),
  Importance = rf_model$variable.importance
) |>
  arrange(desc(Importance))

ggplot(importance_df[1:15, ], aes(x = reorder(Variable, Importance), y = Importance)) +
  geom_col(fill = "#00796b") +
  coord_flip() +
  labs(title = "Top 15 Variable Importances (Random Forest)",
       x = "Variable",
       y = "Importance") +
  theme_minimal(base_size = 14)
```


Top 15 Variable Importances (Random Fore
bmi
gen_hlth
age
high_bp
income
phys_hlth
ment_hlth
education
high_chol
diff_walk
heart_diseaseor_attack
fruits
smoker
sex
veggies
Variable
0
1000
2000
3000
4000
5000
Importance